



UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE

SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO	:	202650 abril – agosto 2026
ASIGNATURA	:	Fundamentos de sistemas web.
TITULO ACTIVIDAD	:	Taller: Construcción de interacciones web
ESTUDIANTE	:	Vasquez Lady
NIVEL-PARALELO – NRC	:	Cuarto- Nrc 29535
DOCENTE	:	Ing. Geovanny Brito Casanova.
FECHA DE ENTREGA	:	17 de julio del 2026.

SANTO DOMINGO – ECUADOR

Link: https://github.com/LadyVasquez22/U3_FWEB_Taller1

Figura 1: Formulario académico con cuatro notas.	4
Figura 2: Resultado de beca calculada según la carrera y el promedio, 60%.	5
Figura 3: Resultado de beca calculada según la carrera y el promedio, 80%.	5
Figura 4: Resultado de beca calculada según la carrera y el promedio, 100%.	6
Figura 5: Mensaje de validación al intentar registrar un nombre repetido.	8
Figura 6: Tabla del historial de estudiantes evaluados.....	9
Figura 7: Búsqueda de estudiante por nombre.	10
Figura 8: Ranking de estudiantes ordenado de mayor a menor promedio.....	11
Figura 9: Objeto final del estudiante presentado en formato JSON.	12

1. Descripción de la actividad

El presente taller consistió en ampliar un evaluador académico desarrollado inicialmente con HTML, JavaScript y Bootstrap. La aplicación base permitía ingresar el nombre, la carrera y tres notas de un estudiante para calcular su promedio y determinar su estado académico.

A partir de esta estructura se agregaron nuevas funcionalidades, como una cuarta nota, ingreso de valores decimales, cálculo de becas, promedio especial, nota mayor y menor, recomendaciones académicas, clasificación del rendimiento, historial de estudiantes, búsqueda por nombre, ranking y estadísticas generales del curso.

Las funcionalidades fueron desarrolladas mediante funciones de JavaScript, eventos, validaciones, arreglos, objetos, condicionales, ciclos y manipulación del modelo DOM. Los resultados se presentan visualmente mediante alertas, botones, tablas, tarjetas y otros componentes de Bootstrap.

2. Objetivos

2.1. Objetivo general

Desarrollar y ampliar un evaluador académico interactivo utilizando JavaScript, HTML y Bootstrap, aplicando funciones, eventos, validaciones, arreglos, objetos, estructuras condicionales, ciclos y manipulación del DOM.

2.2. Objetivos específicos

- Modificar el formulario académico para registrar cuatro notas con valores decimales.
- Implementar funciones para calcular promedios, estados, rendimiento, becas y recomendaciones.
- Almacenar los estudiantes evaluados en un arreglo de objetos.
- Validar los datos ingresados antes de procesarlos y almacenarlos.
- Mostrar el historial de estudiantes mediante una tabla dinámica.
- Crear funciones de búsqueda, ranking y análisis general del curso.

- Presentar los resultados mediante componentes visuales de Bootstrap.
- Mostrar el objeto final del estudiante en formato JSON.

3. Marco conceptual breve

3.1. JavaScript

JavaScript es un lenguaje de programación utilizado para agregar comportamiento e interactividad a las páginas web. En este proyecto se utilizó para leer los datos del formulario, realizar cálculos, validar información y modificar los elementos visibles en la página.

3.2. Funciones

Una función es un bloque de código que realiza una tarea específica. El sistema utiliza funciones para calcular promedios, obtener notas mayores y menores, generar recomendaciones, clasificar estados y mostrar información.

3.3. Eventos

Los eventos permiten ejecutar una acción cuando el usuario interactúa con la página. En este proyecto se utilizó el evento click en los botones de evaluar, limpiar, listar, buscar y mostrar ranking.

3.4. Arreglos

Un arreglo permite almacenar varios elementos en una misma variable. El arreglo global estudiantes guarda todos los estudiantes que han sido evaluados correctamente.

3.5. Objetos

Un objeto agrupa diferentes datos relacionados. Cada estudiante se guarda como un objeto que contiene su nombre, carrera, notas, promedio, estado, rendimiento, beca y recomendación.

3.6. Validaciones

Las validaciones comprueban que los datos ingresados sean correctos. El sistema verifica campos vacíos, longitud del nombre, notas fuera del rango permitido y nombres repetidos.

3.7. Manipulación del DOM

El DOM representa los elementos del documento HTML. JavaScript utiliza métodos como `document.getElementById()` y `document.createElement()` para leer, modificar y crear elementos dentro de la página.

3.8. Bootstrap

Bootstrap es una biblioteca de estilos que permite construir interfaces web adaptables. En este proyecto se utilizaron formularios, botones, alertas, tarjetas, insignias y tablas.

3.9. JSON

JSON es un formato utilizado para representar información estructurada. El objeto del estudiante se convierte a JSON mediante `JSON.stringify()` para mostrarlo dentro de la aplicación.

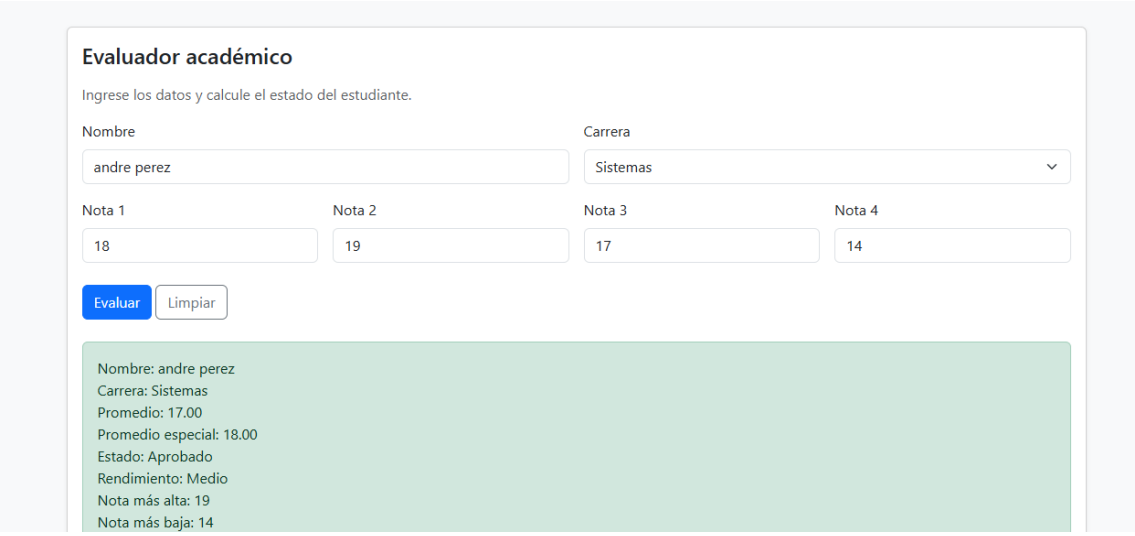
4. Desarrollo paso a paso

4.1. Agregar una cuarta nota al formulario

Se agregó en el archivo `index.html` un nuevo campo numérico identificado como `nota4`. También se modificó JavaScript para leer las cuatro notas mediante un arreglo de identificadores.

Cada nota se obtiene mediante la función `obtenerNota()` y posteriormente se convierte de texto a número.

Comprobación: se ingresaron cuatro notas y el sistema calculó el promedio utilizando todos los valores, como se observa en la Figura 1



The screenshot shows a web form titled "Evaluador académico" with the instruction "Ingrese los datos y calcule el estado del estudiante." The form contains the following fields and results:

- Nombre:** Input field with "andre perez".
- Carrera:** Dropdown menu with "Sistemas" selected.
- Nota 1:** Input field with "18".
- Nota 2:** Input field with "19".
- Nota 3:** Input field with "17".
- Nota 4:** Input field with "14".
- Buttons:** "Evaluar" (blue) and "Limpiar" (white).
- Results Panel (Green background):**
 - Nombre: andre perez
 - Carrera: Sistemas
 - Promedio: 17.00
 - Promedio especial: 18.00
 - Estado: Aprobado
 - Rendimiento: Medio
 - Nota más alta: 19
 - Nota más baja: 14

Figura 1: Formulario académico con cuatro notas.

4.2. Permitir notas con decimales

Los campos de notas fueron configurados con el atributo:

```
step="0.01"
```

Este atributo permite ingresar valores decimales como 15.50, 17.25 o 19.75.

El promedio se presenta con dos posiciones decimales mediante:

```
promedio.toFixed(2)
```

Comprobación: se ingresaron notas decimales y el resultado se mostró correctamente con dos decimales.

4.3. Ajustar las carreras del formulario

El selector de carrera fue modificado para incluir las siguientes opciones, ya que anteriormente en lugar de la opción “Sistemas” estaba la opción “Redes”:

- Tecnologías de la Información.
- Software.
- Sistemas.

Cada opción posee un valor que JavaScript utiliza para aplicar las reglas de beca correspondientes.

4.4. Crear el mensaje de beca académica

Se creó la función `calcularBeca()`, que recibe la carrera y el promedio del estudiante.

Las condiciones aplicadas fueron:

- Tecnologías de la Información y promedio mayor a 18: beca del 100 %.
- Software y promedio mayor a 17: beca del 80 %.
- Sistemas y promedio mayor a 16: beca del 60 %.
- Cuando no se cumple ninguna condición: sin beca académica.

El resultado se muestra en el elemento identificado como `mensajeBeca`.

Beca académica del 60%

Figura 2: Resultado de beca calculada según la carrera y el promedio, 60%.

Nombre: lady vasquez
Carrera: Software
Promedio: 18.25
Promedio especial: 18.67
Estado: Aprobado
Rendimiento: Alto
Nota más alta: 19
Nota más baja: 17
Notas aprobadas: 4
Notas en recuperación: 0
Notas reprobadas: 0
Recomendación: Mantener el desempeño y apoyar a compañeros.

Beca académica del 80%

Figura 3: Resultado de beca calculada según la carrera y el promedio, 80%.

Nombre: arleth muñoz
Carrera: TI
Promedio: 18.75
Promedio especial: 19.00
Estado: Aprobado
Rendimiento: Alto
Nota más alta: 19
Nota más baja: 18
Notas aprobadas: 4
Notas en recuperación: 0
Notas reprobadas: 0
Recomendación: Mantener el desempeño y apoyar a compañeros.

Beca académica del 100%

Figura 4: Resultado de beca calculada según la carrera y el promedio, 100%.

4.5. Mostrar colores según el tipo de beca

El mensaje de beca cambia de color mediante las clases de Bootstrap:

- `alert-success`: beca del **100** %.
- `alert-primary`: beca del **80** %.
- `alert-warning`: beca del **60** %.
- `alert-secondary`: sin beca.

La función `mostrarBeca()` modifica dinámicamente la clase del elemento HTML.

4.6. Calcular el promedio especial

Se creó la función `calcularPromedioEspecial()`. Esta función realiza una copia del arreglo de notas, identifica la nota menor y la elimina utilizando `splice()`.

Después calcula el promedio únicamente con las tres notas restantes.

```
function calcularPromedioEspecial(notas) {  
  const copia = [...notas];  
  copia.splice(copia.indexOf(obtenerNotaMenor(copia)), 1);  
  return calcularPromedio(copia);  
}
```

Se realizó una copia del arreglo para evitar modificar las notas originales del estudiante.

4.7. Mostrar la nota más alta y la nota más baja

Para identificar los valores mayor y menor se crearon las funciones:

```
function obtenerNotaMayor(notas) {  
  return Math.max(...notas);  
}  
  
function obtenerNotaMenor(notas) {  
  return Math.min(...notas);  
}
```

El operador de propagación permite enviar todas las notas a los métodos `Math.max()` y `Math.min()`.

Los resultados se presentan dentro de la alerta individual del estudiante.

4.8. Contar notas aprobadas, en recuperación y reprobadas

Se creó la función contarNotas(), que recorre el arreglo mediante forEach().

La clasificación utilizada fue:

- Nota mayor o igual a 14: aprobada.
- Nota entre 10 y 13.99: recuperación.
- Nota menor a 10: reprobada.

La función devuelve un objeto con los tres conteos y estos valores se incluyen en el resultado individual.

4.9. Generar una recomendación académica

La función generarRecomendacion() analiza el promedio y devuelve un mensaje:

- De 18 a 20: mantener el desempeño y apoyar a compañeros.
- De 14 a 17.99: reforzar temas específicos.
- De 10 a 13.99: asistir a tutorías y practicar ejercicios.
- De 0 a 9.99: repetir contenidos base y solicitar acompañamiento.

La recomendación se guarda dentro del objeto del estudiante.

4.10. Clasificar el rendimiento académico

Se creó la función clasificarRendimiento(), que clasifica al estudiante como:

- Alto: promedio mayor o igual a 18.
- Medio: promedio mayor o igual a 14.
- Básico: promedio mayor o igual a 10.
- Bajo: promedio menor a 10.

Esta clasificación se muestra tanto en el resultado individual como en la tabla del historial.

4.11. Crear el historial de estudiantes

Se declaró el siguiente arreglo global:

```
const estudiantes = [];
```

Cada vez que un estudiante supera todas las validaciones, su objeto se agrega mediante:

```
estudiantes.push(estudiante);
```

El arreglo permanece disponible mientras la página se encuentra abierta.

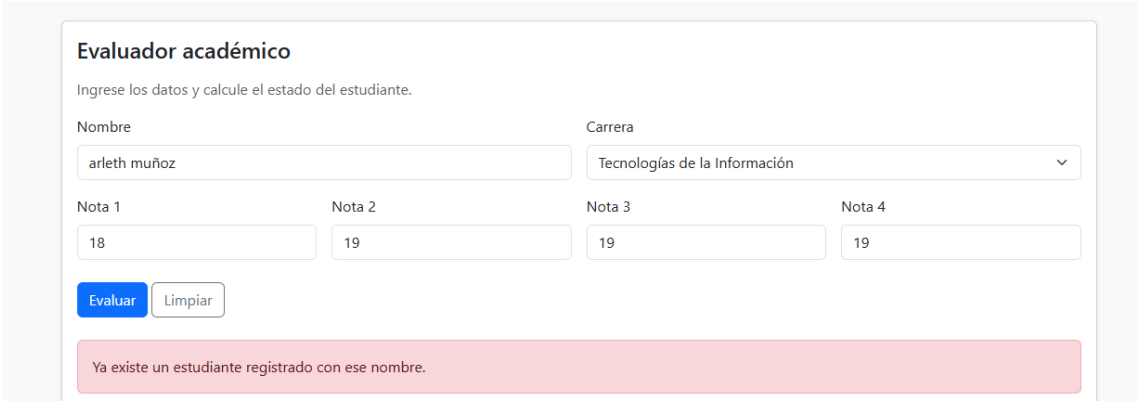
4.12. Validar nombres repetidos

Antes de agregar un estudiante se ejecuta la función `nombreRepetido()`.

Para realizar una comparación correcta se creó también `normalizarNombre()`, que:

- Elimina espacios al inicio y al final.
- Reemplaza espacios repetidos.
- Convierte el texto a minúsculas.

Esto evita registrar dos veces nombres como “Arleth Muñoz”, “arleth muñoz” o “ARLETH MUÑOZ”.



The screenshot shows a web form titled "Evaluador académico" with the instruction "Ingrese los datos y calcule el estado del estudiante." The form contains the following fields: "Nombre" (text input with "arleth muñoz"), "Carrera" (dropdown menu with "Tecnologías de la Información"), "Nota 1" (18), "Nota 2" (19), "Nota 3" (19), and "Nota 4" (19). Below the inputs are "Evaluar" and "Limpiar" buttons. A red error message at the bottom states: "Ya existe un estudiante registrado con ese nombre."

Figura 5: Mensaje de validación al intentar registrar un nombre repetido.

4.13. Crear el botón para listar estudiantes

Se agregó el botón `btnListar`, conectado al evento `click`.

```
btnListar.addEventListener("click", () => mostrarTabla(estudiantes));
```

La función `mostrarTabla()` recorre el arreglo de estudiantes y crea dinámicamente las filas y celdas de la tabla.

Para crear los elementos se utilizaron:

```
document.createElement("tr");  
document.createElement("td");
```


Historial del curso						Total: 3
Nombre del estudiante						Buscar estudiante
Listar estudiantes Mostrar ranking						
Aprobados: 3 Recuperación: 0 Reprobados: 0 Promedio general: 18.00 Mejor promedio: arleth muñoz (18.75) Menor promedio: andre perez (17.00)						
#	Nombre	Carrera	Promedio	Estado	Rendimiento	Beca
1	andre perez	Sistemas	17.00	Aprobado	Medio	Beca académica del 60%
2	lady vasquez	Software	18.25	Aprobado	Alto	Beca académica del 80%
3	arleth muñoz	TI	18.75	Aprobado	Alto	Beca académica del 100%

Figura 6: Tabla del historial de estudiantes evaluados.

4.14. Mostrar el total de estudiantes evaluados

Se agregó una insignia de Bootstrap con el identificador totalEstudiantes.

Después de registrar un estudiante, la función actualizarResumenCurso() cambia su contenido:

```
document.getElementById("totalEstudiantes").textContent = `Total: ${total}`;
```

De esta manera el total aumenta automáticamente con cada registro.

4.15. Contar los estados académicos del curso

La función actualizarResumenCurso() recorre el historial y cuenta cuántos estudiantes están:

- Aprobados.
- En recuperación.
- Reprobados.

Para almacenar el conteo se utiliza el objeto:

```
const estados = {
  Aprobado: 0,
  Recuperación: 0,
  Reprobado: 0
};
```

Luego se incrementa la propiedad correspondiente según el estado de cada estudiante.

4.16. Calcular el promedio general del curso

El promedio general se calcula sumando los promedios individuales de todos los estudiantes.

```
const promedioCurso = estudiantes.reduce(  
  (suma, estudiante) => suma + estudiante.promedio,  
  0  
)/ total;
```

El método `reduce()` acumula los promedios registrados y finalmente la suma se divide para el total de estudiantes.

4.17. Buscar estudiante por nombre

Se agregó un campo de búsqueda y el botón `btnBuscar`.

Al presionar el botón se ejecuta la función `buscarEstudiante()`, que utiliza el método `find()` para localizar un estudiante.

La búsqueda utiliza el nombre normalizado, por lo que no diferencia entre mayúsculas y minúsculas.

Cuando se encuentra el estudiante se muestra su nombre, promedio, estado y beca.

The screenshot shows a web interface titled 'Historial del curso' with a 'Total: 3' badge. It features a search bar containing 'lady vasquez' and a 'Buscar estudiante' button. Below the search bar are two buttons: 'Listar estudiantes' (green) and 'Mostrar ranking' (light green). A green box displays the search result: 'lady vasquez | Promedio: 18.25 | Estado: Aprobado | Beca académica del 80%'. A light blue box shows summary statistics: 'Aprobados: 3 | Recuperación: 0 | Reprobados: 0 | Promedio general: 18.00 | Mejor promedio: arleth muñoz (18.75) | Menor promedio: andre perez (17.00)'. At the bottom, a table lists the student details.

#	Nombre	Carrera	Promedio	Estado	Rendimiento	Beca
1	lady vasquez	Software	18.25	Aprobado	Alto	Beca académica del 80%

Figura 7: Búsqueda de estudiante por nombre.

4.18. Crear el ranking de estudiantes

La función `mostrarRanking()` crea una copia del historial y la ordena desde el promedio más alto hasta el menor.

```
const ranking = [...estudiantes].sort(  
  (a, b) => b.promedio - a.promedio  
);
```

Se utiliza una copia para evitar modificar el orden original del arreglo.

lady vasquez | Promedio: 18.25 | Estado: Aprobado | Beca académica del 80%

Aprobados: 3 | Recuperación: 0 | Reprobados: 0 | Promedio general: 18.00 | Mejor promedio: arleth Muñoz (18.75) | Menor promedio: andre perez (17.00)

#	Nombre	Carrera	Promedio	Estado	Rendimiento	Beca
1	arleth Muñoz	TI	18.75	Aprobado	Alto	Beca académica del 100%
2	lady vasquez	Software	18.25	Aprobado	Alto	Beca académica del 80%
3	andre perez	Sistemas	17.00	Aprobado	Medio	Beca académica del 60%

Figura 8: Ranking de estudiantes ordenado de mayor a menor promedio.

4.19. Mostrar el estudiante con mayor y menor promedio

Después de ordenar los estudiantes, el primer elemento representa al estudiante con mejor promedio y el último al estudiante con menor promedio.

```
const mejor = ordenados[0];
```

```
const menor = ordenados[ordenados.length - 1];
```

Estos datos se muestran en el resumen general del curso junto con sus respectivos promedios.

6.20 Mejorar el objeto final del estudiante

El objeto final contiene todos los datos solicitados:

```
const estudiante = {  
  nombre,  
  carrera,  
  notas,  
  promedio,  
  promedioEspecial,  
  notaMayor,  
  notaMenor,  
  estado,  
  rendimiento,  
  beca,  
  recomendacion  
};
```

Después, el objeto se transforma a JSON mediante:

```
JSON.stringify(estudiante, null, 2);
```

El segundo parámetro se establece como null y el tercero como 2 para mostrar el contenido con una indentación legible.

```
{
  "nombre": "arleth muñoz",
  "carrera": "TI",
  "notas": [
    18,
    19,
    19,
    19
  ],
  "promedio": 18.75,
  "promedioEspecial": 19,
  "notaMayor": 19,
  "notaMenor": 18,
  "estado": "Aprobado",
  "rendimiento": "Alto",
  "beca": "Beca académica del 100%",
  "recomendación": "Mantener el desempeño y apoyar a compañeros."
}
```

Figura 9: Objeto final del estudiante presentado en formato JSON.

Criterio personal

Durante el desarrollo de este taller aprendí a relacionar los elementos de un formulario HTML con funciones de JavaScript y a mostrar los resultados directamente en la página mediante la manipulación del DOM.

La parte que me resultó más compleja fue trabajar con el historial de estudiantes, debido a que fue necesario almacenar objetos dentro de un arreglo y luego recorrerlos para calcular el promedio general, contar los estados académicos, realizar búsquedas y ordenar el ranking.

La funcionalidad que considero más útil es el historial, porque permite observar la información de todos los estudiantes y obtener un resumen general del curso. También considero importante la validación de nombres repetidos, ya que evita que una misma persona sea registrada más de una vez.

Este taller me ayudó a comprender que JavaScript no solamente permite realizar cálculos, sino también controlar eventos, validar datos, crear objetos, trabajar con arreglos y modificar dinámicamente los componentes de una interfaz web.